

CPU Scheduling In Operating System (OS)

Scheduling is the process of deciding which process will run on the CPU at a given time when there are multiple processes waiting to execute.

- Process Scheduling is a fundamental function or task of OS. When a computer is multi programmed, it has multiple processes in main memory & get executed towards the CPU one after another. If there is only one CPU is available, then a choice has to be made regarding which process to execute next. This decision making of process is known as “Process scheduling”.
- Process Scheduling is an Operating System task that schedules processes of different states like ready, waiting, and running.
- Process scheduling allows OS to allocate a time interval for each process that will execute by CPU.
- Another important reason for using a process scheduling system is that it keeps the CPU busy all the time. This also allows you to get the minimum response time for programs.
- Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Definition

Scheduling is the method by which the OS selects the next process from the ready queue and allocates the CPU (or other resources) to it.

Goals of Scheduling (scheduling criteria)

- Maximize CPU utilization (keep CPU busy).
- Maximize throughput (number of processes completed per unit time).
- Minimize turnaround time (time taken to execute a process).
- Minimize waiting time (time spent in ready queue).
- Minimize response time (time taken to start responding).
- Ensure fairness (all processes get CPU time).

Types of Scheduler

1. Long-Term Scheduling

- Decides which jobs from the job pool in secondary storage are brought into main memory for execution.
- Controls the degree of multiprogramming.

2. Medium-Term Scheduling

- Temporarily removes (suspends) processes from memory and later resumes them.
- Used to improve process mix and free up memory.

3. Short-Term Scheduling (CPU Scheduling)

- Decides which process in the ready queue will get CPU next.
- Very frequent, happens in milliseconds.

CPU Scheduling Algorithms

What is FCFS?

First-Come, First-Served (FCFS) is the simplest CPU scheduling algorithm where the process that arrives first gets the CPU first. It is non-preemptive, meaning once a process starts executing, it runs until completion.

- In the "First come first serve" scheduling algorithm, as the name suggests, the process which arrives first, gets executed first, or we can say that the process which requests the CPU first, gets the CPU allocated first.
- This is the simplest CPU scheduling algorithm. In this scheme, the process which requests the CPU first, that is allocated to the CPU first. The implementation of the FCFS algorithm is easily managed with a FIFO queue. When a process enters the ready queue its PCB is linked on to the rear of the queue.
- First Come First Serve, is just like FIFO (First in First out) Queue data structure, where the data element which is added to the queue first, is the one who leaves the queue first.
- The FCFS is used in Batch Systems.
 - A perfect real life example of FCFS scheduling is buying tickets at ticket counter.

The average waiting time under FCFS policy is quite long.

Feature Description

Type	non-preemptive
Scheduling Basis	Arrival Time (AT)
Fairness	High (First in, first served)
Starvation	No
Implementation	FIFO (Queue based)
Ideal For	Batch systems

Important Formulas

Completion Time = Arrival time + Burst time + Waiting time

Turnaround Time = Completion Time - Arrival time

Waiting Time = Turnaround Time - Burst time

Avg Turnaround Time = Σ Total of turnaround time / No. of process

Avg Waiting Time = Σ Total of Waiting time / No. of process

1)Example:

Queue at a Bank casher Counter

- Customers arrive at the bank and form a single line (queue) for the cash counter.
- The casher serves **the first customer who arrives first**, then the next, and so on.
- No customer is skipped or given priority – service is strictly in the order of arrival.

How it matches FCFS scheduling:

1. **Arrival order = Execution order** (like processes arriving in the ready queue).
2. **No preemption** – once a casher starts serving a customer, they finish before moving to the next.
1. **Waiting time depends on earlier customers** – if the first customer takes a long time, others wait longer (similar to the **convoy effect** in FCFS).

Example scenario:

- Customer A arrives at 10:00 (requires 5 minutes).
- Customer B arrives at 10:02 (requires 2 minutes).
- Customer C arrives at 10:03 (requires 3 minutes).

Execution order (FCFS): A → B → C

- Even though B and C require less time, they must wait until A finishes.

Advantages of FCFS Scheduling

1. **Simple to understand and implement** – uses a basic FIFO queue.
2. **Fairness** – every process gets a chance in the order it arrives.
3. **No starvation** – all processes will eventually execute.
4. **Low overhead** – no need for complex context switching.
5. **Best suited for batch systems** or systems with similar process sizes.

Disadvantages of FCFS Scheduling

1. **Poor performance with varying burst times** – long jobs delay short ones.
2. **Convoy effect** – short processes get stuck behind long ones.
3. **Not suitable for time-sharing systems** – lacks responsiveness.

Example 1

Given: Process	AT	BT
P1	0	4
P2	1	3
P3	2	1
P4	3	2
P5	4	5

Gantt Chart:

P1	P2	P3	P4	P5
0	4	7	8	10
15				

Calculations:

Process	AT	BT	CT	TAT = CT-AT	WT = TAT-BT
P1	0	4	4	4	0
P2	1	3	7	6	3
P3	2	1	8	6	5
P4	3	2	10	7	5
P5	4	5	15	11	6

Average TAT = $(4+6+6+7+11) / 5 = 6.8$
 $= 3.8$

Average WT = $(0+3+5+5+6) / 5$

Example 2

Given:

Process	AT	BT
P1	2	6
P2	5	2
P3	1	8
P4	0	3
P5	4	4

Gantt Chart:

P4	P3	P1	P5	P2
0	3	11	17	21
				23

Calculations:

Process	AT	BT	CT	TAT	WT
P4	0	3	3	3	0
P3	1	8	11	10	2
P1	2	6	17	15	9
P5	4	4	21	17	13
P2	5	2	23	18	16

Average TAT = 12.6
 WT = 8.0

Average

Shortest Job First (SJF)

What is SJF?

Shortest Job First (SJF) is a CPU scheduling algorithm that selects the process with the smallest burst time first.

- The idea behind the SJF algorithm is to select or pick the quickest fastest little job first, this shortest or little job is get selected first from ready queue and scheduled towards the CPU and CPU execute it till its burst time. After that next smallest fastest job is selected and scheduled towards the CPU and remaining process is done in the same way... (Technically this algorithm picks a process based on the next shortest CPU burst, not the overall process time.)
- This algorithm associates with each process if the CPU is available. This scheduling is also known as shortest next CPU burst, because the scheduling is done by examining the length of the next CPU burst of the process rather than its total length.

It can be **non-preemptive** (standard SJF) or **preemptive** (Shortest Remaining Time First).

Feature	Description
Type	Non-preemptive
Scheduling Basis	Burst Time
Fairness	Medium
Starvation	Yes (long processes may starve)
Implementation	Process with minimum burst time is selected
Ideal For	Batch systems with predictable burst times

Real-Life Example

In a **printing shop**, smaller print jobs are completed first.

- Short documents are handled quickly, reducing waiting time for small tasks.
- Large print jobs may need to wait longer.

Advantages

1. Minimizes average waiting time.
2. Efficient for batch processing.
3. Simple to understand and implement.

Disadvantages

1. Starvation of longer processes.
2. Requires knowing burst time in advance.
3. Not suitable for real-time systems.

Given:

Process	AT	BT
P1	0	6
P2	2	8
P3	4	7
P4	5	3

Gantt Chart:

P1					P2					P3					P4									
0					6					9					16					24				

Calculations:

Process	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P4	5	3	9	4	1
P3	4	7	16	12	5
P2	2	8	24	22	14

Average TAT = 11

Average WT = 5

Shortest Remaining Time First (SRTF)

What is SRTF?

Shortest Remaining Time First (SRTF) is the **preemptive version of SJF**. At any time, the process with the **smallest remaining burst time** is scheduled.

- The preemptive SJF is also known as “Shortest Remaining Time First”.
- This Algorithm is the preemptive version of SJF scheduling. In SRTF, the execution of the process can be stopped after certain amount of time (say unit interval of time). At the arrival of every process in ready queue, the short term scheduler schedules the process with the least remaining burst time among the list of available processes and the running process.

Once all the processes are available in the ready queue, No preemption will be done and the algorithm will work as SJF scheduling. The context of the process is saved in the Process Control Block when the process is removed from the execution and the next process is scheduled. This PCB is accessed on the next execution of this process.

Feature	Description
Type	Preemptive
Scheduling Basis	Remaining Burst Time
Fairness	Medium
Starvation	Yes (long processes may starve)
Implementation	CPU is preempted if a new process has shorter burst time
Ideal For	Time-sharing systems with short processes

Real-Life Example

In a **hospital emergency room**, patients with the most urgent (shortest treatment needed) are treated first. A patient requiring only a quick treatment may be attended before someone with a long operation.

Example Scenario

Given:

Process	AT	BT
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Gantt Chart:

P1		P2		P3		P2		P4		P1	
0	2	4	5	7	11	14					

Calculations:

Process	AT	BT	CT	TAT	WT
P1	0	7	14	14	7
P2	2	4	7	5	1
P3	4	1	5	1	0
P4	5	4	11	6	2

Average TAT = 6.5

Average WT = 2.5

Round Robin (RR)

What is RR?

Round Robin (RR) is a **preemptive scheduling algorithm** where each process gets a fixed **time quantum**.

When the time quantum expires, the process is preempted and placed at the end of the queue.

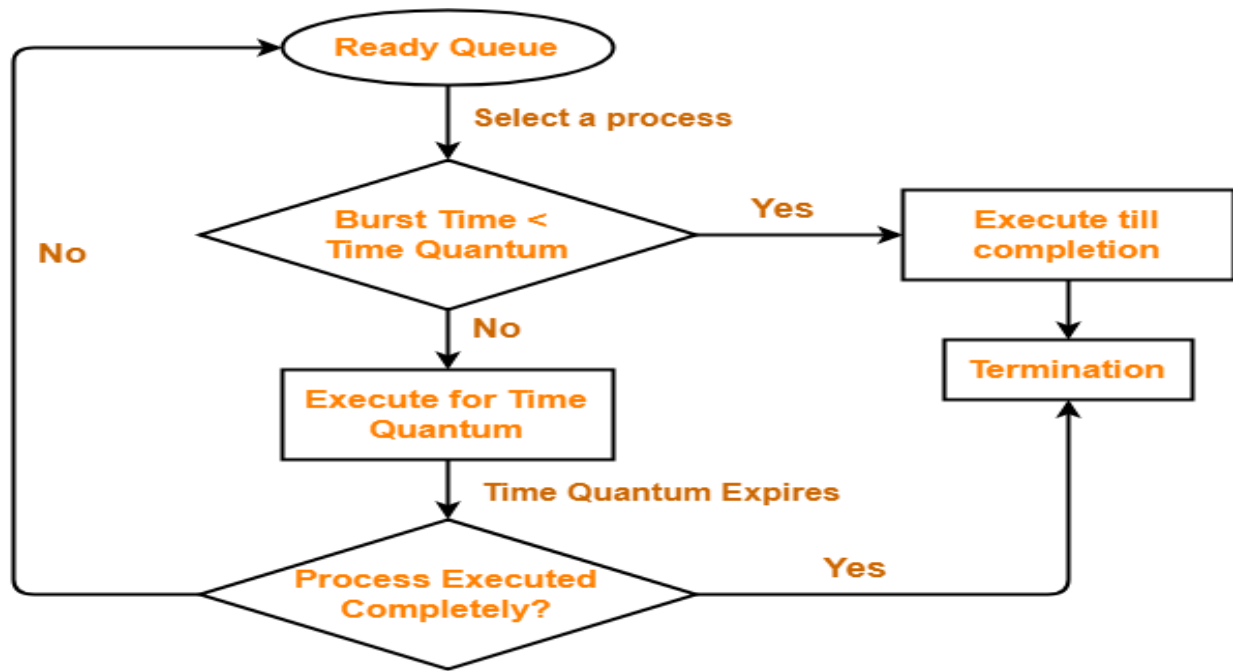
- This type of algorithm is designed only for the time sharing system i.e. multitasking operating system.
- It is similar to FCFS scheduling with preemption condition to switch between processes.
- In this algorithm, a small unit of time called 'quantum time' or 'time slice' is used to switch between the processes.

- It would give deterministic response time to each process. Therefore this algorithm is best in case of average response time for process.
 - The average waiting time under the round robin policy is also quite long.
 - While constructing Gantt chart, maintain ready queue also to avoid mistake.
- Since ready queue is queue that has FIFO sequence to add & remove processes.

Process of Round Robin scheduling:

- CPU is assigned to the process on the basis of FCFS for a fixed amount of time (Time quantum)
- After the time quantum expires, the running process is preempted and sent to the ready queue.
- Then, the processor is assigned to the next arrived process. And remaining process is done on FCFC with preemption basis.

The overall process of Round Robin scheduling algorithm is shown in following



Round Robin Scheduling

chart:

Feature	Description
Type	Preemptive
Scheduling Basis	Time Quantum (fixed)
Fairness	High

Feature	Description
Starvation	No
Implementation	Circular queue
Ideal For	Time-sharing, multitasking systems

Advantages

1. Fair – every process gets equal share of CPU.
2. Suitable for time-sharing and interactive systems.
3. No starvation.

Disadvantages

1. High average turnaround time for small quantum.
2. Context switching overhead.
3. Performance depends heavily on time quantum choice.

Real-Life Example

In a **call center help desk**, each caller is given a fixed time slot to talk.

If their issue is not resolved in that time, the call is placed back in the queue until their next turn.

Example Scenario

Given (Time Quantum = 2):

Process	AT	BT
P1	0	5,3,1,0(Remaining)
P2	1	4,2,0(Remaining)
P3	2	2
P4	3	1

Ready Queue:

P1	P2	P3	P1	P4	P2	P1
----	----	----	----	----	----	----

Gantt Chart:

P1	P2	P3	P1	P4	P2	P1	
0	2	4	6	8	9	11	12

Calculations:

Process	AT	BT	CT	TAT	WT
P1	0	5	12	12	7
P2	1	4	11	10	6
P3	2	2	6	4	2
P4	3	1	7	4	3

Average TAT = 7.5

Average WT = 4.5

Priority Scheduling

What is Priority Scheduling? In Priority Scheduling, each process is assigned a **priority number**.

CPU is allocated to the process with the **highest priority**.

It Priority Scheduling Algorithm:

Priority scheduling is a more general case of SJF, in which each job is assigned a priority and the job with the highest priority gets scheduled first.

- In this scheduling a priority is associated with each process and the CPU is allocated to the process with the highest priority. Equal priority processes are scheduled in FCFS manner.
- Priorities can be assigned either internally or externally. Internal priorities are assigned by the OS using criteria such as average burst time, ratio of CPU to I/O activity, system resource use, and other factors available to the kernel. External priorities are assigned by users, based on the importance of the job, fees paid, politics, etc.
- Note that in practice, priorities are implemented using integers within a fixed range, but there is no agreed-upon convention as to whether "high" priorities

use large numbers or small numbers. Here, we consider lesser the number having higher priority.

- Priority scheduling can be either preemptive or non-preemptive.

Feature	Description
Type	Preemptive / non-preemptive
Scheduling Basis	Priority number (lower number = higher priority)
Fairness	Low (depends on priority)
Starvation	Yes (low-priority processes may starve)
Implementation	Priority Queue
Ideal For	Systems where certain tasks must finish first

Real-Life Example

In an **airport boarding system**, first-class passengers (high priority) board before economy-class passengers (low priority).

Advantages

1. Important tasks handled first.
2. Flexible – priorities can be dynamically changed.
3. Useful in real-time systems.

Disadvantages

1. Starvation for low-priority tasks.
2. Difficult to decide priority objectively.
3. Aging must be used to avoid starvation.
4. Priority scheduling Algorithm (Non-preemptive case):
5. In this scheduling a priority is associated with each process and the CPU is allocated to the process with the highest priority. Equal priority processes are scheduled in FCFS manner.

Consider the following example:

Process	Priority	Arrival Time	Burst Time
P1	3	0	8
P2	4	1	2
P3	4	3	4
P4	5	4	1
P5	2	5	6
P6	6	6	5
P7	1	10	1

Using Priority scheduling algorithm (Non-preemptive case) find the complete time, waiting time, turnaround time, average waiting time and average turnaround time. (Consider here, lesser the number in priority column have higher priority)

Solution:

According to the priority scheduling the Gantt chart will be:

P1	P5	P7	P2	P3	P4	P6	
0	8	14	15	17	21	22	27

Complete time for process P1=8, P2=17, P3=21, P4=22, P5=14, P6=27, P7=15

Turnaround time for process P1=8, P2=16, P3=18, P4=18, P5=9, P6=21, P7=5 (Since, TAT= CT -AT)

Waiting time for process P1=0, P2=14, P3=14, P4=17, P5=3, P6=16, P7=4 (Since, WT=TAT- CPU Time)

Average turnaround time= $(8+16+18+18+9+21+5)/7=95/7=13.57$

Average waiting time= $(0+14+14+17+3+16+4)/7=68/7= 9.71$

- Priority scheduling Algorithm (Preemptive case):
- In Preemptive Priority Scheduling, at the time of arrival of a process in the ready queue, its Priority is compared with the priority of the other processes present in the ready queue as well as with the one which is being executed by the CPU at

that point of time. The One with the highest priority among all the available processes will be given the CPU next.

- In this algorithm the execution of the process can also be stopped after certain amount of time (say unit interval of time). At the arrival of every process in ready queue, the short term scheduler schedules the process with the higher priority process among the list of available processes and the running process.
- The difference between preemptive priority scheduling and non-preemptive priority scheduling is that, in the preemptive priority scheduling, the process which is being executed can be stopped at the arrival of a higher priority job.
- Once all the processes get available in the ready queue, then algorithm will behave as non-preemptive priority scheduling, which means the process scheduled will run till the completion and no preemption will be done.

Consider Following example:

Process Id	Priority	Arrival Time	Burst Time
P1	2	0	1
P2	6	1	7
P3	3	2	3
P4	5	3	6
P5	4	4	5
P6	10	5	15
P7	9	15	8

Using Priority scheduling algorithm (Preemptive case) find the complete time, waiting time, turnaround time, average waiting time and average turnaround time. (Consider here, lesser the number in priority column have higher priority)

Solution:

According to the priority scheduling the Gantt chart will be:

	P 1	P 2	P 3	P 3	P 3	P 5	P 5	P 5	P 5	P 5	P 4	P 4	P 4	P 4	P 4	P 4	P 2	P 7	P 6
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	22	30	45

Complete time for process P1=1, P2=22, P3=5, P4=16, P5=10, P6=45, P7=30

Turnaround time for process P1=1, P2=21, P3=3, P4=13, P5=6, P6=40, P7=15

(Since, TAT= CT - AT)

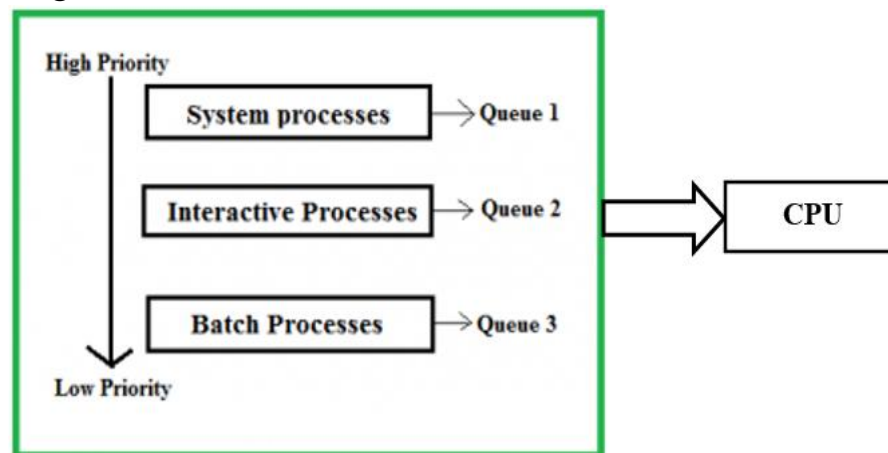
Waiting time for process P1=0, P2=14, P3=0, P4=7, P5=1, P6=25, P7=7 (Since, WT=TAT- CPU Time)

Average turnaround time= $(1+21+3+13+6+40+15)/7=99/7=14.14$

Average waiting time= $(0+14+0+7+1+25+7)/7=54/7= 7.71$

Multilevel Queue Scheduling Algorithm:

- We know that in computer system, there are different kinds of processes (foreground or back ground) were runs like system processes (interrupt), interactive processes (dealing with application software like MSWord, MS Excel etc.) and batch processes.
- It may happen that these processes in the ready queue can be divided into different classes where each class has its own scheduling needs. For example, entire ready queue may be divided into a system process, an interactive process and batch processes classes. These three classes have different scheduling needs. For this kind of situation 'Multilevel Queue Scheduling' is used.
- Ready Queue is divided into separate queues for each class of processes shown in below figure.



- All three different type of processes have their own queue. Each queue have its own Scheduling algorithm and it is implemented by programmer. For example, queue 1 may use SJF, queue 2 may uses Round Robin while queue 3 can use FCFS to schedule there processes.

Scheduling among the queues:

What will happen if all the queues have some processes? Which process should get the CPU To determine this Scheduling among the queues is necessary. There are two ways to do so –

1) Fixed priority preemptive scheduling method – Each queue has absolute priority over lower priority queue. Let us consider following priority order queue 1 > queue

2 > queue 3. According to this algorithm no process in the batch queue (queue 3) can run unless queue 1 and 2 are empty. If any batch process (queue 3) is running and any system (queue 1) or Interactive process (queue 2) entered the ready queue the batch process is preempted.

2) Time slicing – In this method each queue gets certain portion of CPU time and can use it to schedule its own processes. For instance, queue 1 takes 50 percent of CPU time queue 2 takes 30 percent and queue 3 gets 20 percent of CPU time.

Example: Consider below table of four processes under multilevel queue scheduling. Queue number denotes the queue of the process.

Process	Arrival Time	CPU Burst Time	Queue Number
P1	0	4	1
P2	0	3	1
P3	0	8	2
P4	10	5	1

Using Multi-level queue scheduling algorithm find the complete time, waiting time, turnaround time, average waiting time and average turnaround time. By considering Priority of queue 1 is greater than queue 2. (Since, queue 1 is of system processes and queue 2 is of interactive processes)

Also, consider, queue 1 uses Round Robin (Time Quantum = 2) and queue 2 uses FCFS.

Solution:

The resultant Gantt chart of the problem:

P1	P2	P1	P2	P3	P4	P4	P4	P3	
0	2	4	6	7	10	12	14	15	20

At starting both queues have process so process in queue 1 (P1, P2) runs first (because of higher priority) in the round robin fashion and completes after 7 units then process in queue 2 (P3) starts running (as there is no process in queue 1) but while it is running P4 comes in queue 1 and interrupts P3 and start running for 5 second and after its completion P3 takes the CPU and completes its execution.

Complete time for process P1=6, P2=7, P3=20, P4=15

Turnaround time for process P1=6, P2=7, P3=20, P4=5 (Since, TAT= CT - AT)

Waiting time for process P1=2, P2=4, P3=12, P4=0 (Since, WT=TAT- CPU

Time)

Average turnaround time= $(6+7+20+5)/4=38/4=9.5$

Average waiting time= $(2+4+12+0)/4=18/4= 4.5$

Advantages Multilevel Queue scheduling algorithm:

- The processes are permanently assigned to the queue, so it has advantage of low scheduling overhead.

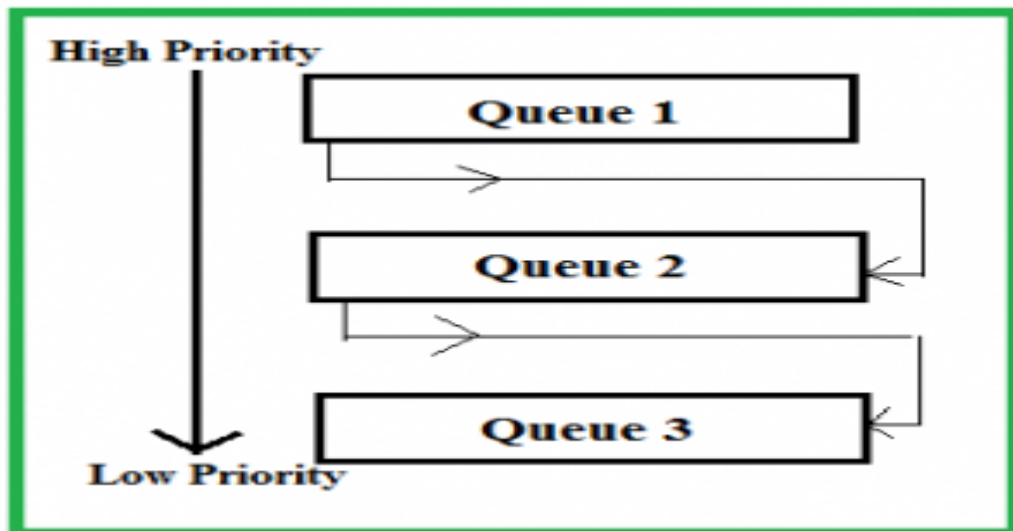
Disadvantages Multilevel Queue scheduling algorithm:

- Some processes may starve for CPU if some higher priority queues are never becoming empty.
- It is inflexible in nature.

Multilevel Feedback Queue Scheduling (MLFQ) CPU Scheduling is like Multilevel Queue (MLQ) Scheduling but in this process can move between the queues. And thus, much more efficient than multilevel queue scheduling.

Characteristics of Multilevel Feedback Queue Scheduling:

- In a multilevel queue-scheduling algorithm, processes are permanently assigned to a queue on entry to the system, and processes are allowed to move between queues.
- As the processes are permanently assigned to the queue, this setup has the advantage of low scheduling overhead,



Multiple queues: Similar to MLQ scheduling, MLFQ scheduling divides processes into multiple queues based on their priority levels. However, unlike MLQ scheduling, processes can move between queues based on their behavior and needs.

Priorities adjusted dynamically: The priority of a process can be adjusted dynamically based on its behavior, such as how much CPU time it has used or how often it has been blocked. Higher-priority processes are given more CPU time and lower-priority processes are given less.

Time-slicing: Each queue is assigned a time quantum or time slice, which determines how much CPU time a process in that queue is allowed to use before it is preempted and moved to a lower priority queue.

Feedback mechanism: MLFQ scheduling uses a feedback mechanism to adjust the priority of a process based on its behavior over time. For example, if a process in a lower-priority queue uses up its time slice, it may be moved to a higher-priority queue to ensure it gets more CPU time.

Preemption: Preemption is allowed in MLFQ scheduling, meaning that a higher-priority process can preempt a lower-priority process to ensure it gets the CPU time it needs.

Multilevel Feedback Queue Scheduling (MLFQ) dynamically adjusts the priority of processes based on their behavior. Understanding how MLFQ works and how to implement it effectively is essential for operating system studies. This complex scheduling mechanism is often examined in competitive exams.

Advantages of Multilevel Feedback Queue Scheduling:

- It is more flexible.
- It allows different processes to move between different queues.
- It prevents starvation by moving a process that waits too long for the lower priority queue to the higher priority queue.

Disadvantages of Multilevel Feedback Queue Scheduling:

- The selection of the best scheduler, it requires some other means to select the values.
- It produces more CPU overheads.
- It is the most complex algorithm.

Example 1: MLFQ with 3 Queues (Q0, Q1, Q2)

Assumptions:

- **Q0** – RR with time quantum = 4
- **Q1** – RR with time quantum = 8
- **Q2** – FCFS

- No aging for this example.
- All processes arrive at time 0.

Process	Arrival Time	Burst Time
P1	0	10
P2	0	4
P3	0	18

Step-by-Step Execution:

Round 1: All processes in Q0

- **P1**: Runs for 4 (Q0) → Remaining = 6 → Moves to Q1
- **P2**: Runs for 4 → Done
- **P3**: Runs for 4 → Remaining = 14 → Moves to Q1

Q1 (RR, tq = 8)

- **P1**: Runs for 6 (remaining < tq) → Done
- **P3**: Runs for 8 → Remaining = 6 → Moves to Q2

Q2 (FCFS)

- **P3**: Runs for 6 → Done

P1	P2	P3	P1	P3	P3	
0	4	8	12	18	26	32

Turnaround Time (TAT) & Waiting Time (WT):

Process	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	10	14	14	4
P2	4	8	8	4
P3	18	32	32	14